

計算がアルゴリズム全体の計算量を支配することもあります。何の工夫もない逆行列計算の計算量は行列のサイズ M に対して $O(M^3)$ です。例えば、最小2乗回帰では、逆行列を求めるべきカーネル行列のサイズはデータ数に比例し、大規模データの処理を逆行列演算が律速します^{*17}。

最適化やモデルのバリエーション解析、解釈性の解析などに使われるヘッセ行列

$$H(f)(\mathbf{x}) = \nabla_{\mathbf{x}}^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \frac{\partial^2 f}{\partial x_N \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}$$

の計算量はパラメータ数 N に対して $O(N^3)$ であり、深層学習などパラメータ数が多い場合には問題になります（必要なメモリは $O(N^2)$ ）。ただし、ヘッセ行列そのものではなく、ヘッセ行列とベクトルの積を知りたい場合にはクリロフ法で効率的に計算できます。

線形代数演算に加え最適化問題にも注意が必要です。機械学習には連続最適化がよく現れますが、グラフ表現や組合せ最適化問題などの形で離散最適化もしばしば必要です。問題が凸計画（＝目的関数が凸関数で、実行可能領域も凸集合）の時には、勾配降下法などにより効率的に大域的最適解を見つけ出せます。それに対して非凸計画の場合には、工夫のないプログラムは局所最適解にハマってしまい、大域的最適解（真の解）になかなかたどり着けません。

計算量の観点からは、凸計画問題で定式化できるのが必ずしも嬉しいわけではありません。例えば、サポートベクタマシン（Support Vector Machine, SVM）の最適化は2次凸計画問題ですが、訓練データセットのサイズ n に対して時間計算量は $O(n^3)$ 、空間計算量は $O(n^2)$ で、大規模高次元データに対する計算は大変です^{69, 70}。一方、深層学習のパラメータ最適化は大抵の場合非凸計画問題ですが、よく用いられる確率的勾配降下法（Stochastic Gradient Descent, SGD）の計算量は $O(1)$ でデータサイズの影響を受けません^{*18}。SGDが出す解は大域的最適解ではなく近似解ですが、良い精度の解が求まるので多用されます。

*17 例えば、1個抜き交差確認法でモデル選択をする場合には n 個のデータすべてに対して逆行列計算が必要で、モデル選択全体の計算量は $O(n^4)$ になってしまいます。

*18 1回のパラメータ更新の計算量です。精度 ϵ の最適解を得るためには $O(1/\epsilon)$ 回反復が必要です。

4.4.2 量子コンピュータで逆行列を高速に求める

逆行列の計算は機械学習のさまざまな場面に登場します。効率的に逆行列を求める手法として、量子位相推定サブルーチンを使うホロー・ハシディム・ロイド（Harrow-Hassidim-Lloyd）のアルゴリズム（HHL アルゴリズム）を紹介しましょう⁷¹。このアルゴリズムは $N \times N$ のエルミート行列 A と N 次元ベクトル $\mathbf{b}$ について

$$A\mathbf{x} = \mathbf{b}$$

という線形方程式の解 $\mathbf{x}$ を求める計算量 $O(s\kappa \text{ poly} \log(s\kappa/\epsilon))$ の量子アルゴリズムです。ここで、 s は行列 A の非ゼロ要素の割合、 κ は行列 A の条件数^{*19}、 ϵ は精度です。HHL アルゴリズムの流れは以下のとおりです（図4.10）。

- i) $\mathbf{b}$ を振幅エンコーディングした $|b\rangle := \sum_i b_i |i\rangle$ を用意する。
- ii) 制御 e^{iAt} ゲートを使う量子位相推定サブルーチンにより A の固有値 λ_i をレジスタ量子ビットに取り出し、 $\sum_i \beta_i |\lambda_i\rangle |u_i\rangle$ を作る（ β_i は $|b\rangle$ を λ_i に対応する固有ベクトル $|u_i\rangle$ で $|b\rangle = \sum_i \beta_i |u_i\rangle$ と展開したときの展開係数）。
- iii) 2.7.2 項の方法と同様に、 $|\lambda_i\rangle$ を制御部とする制御回転ゲートを補助量子ビットに作用させ $\lambda_i^{-1} |0\rangle + \sqrt{1 - \lambda_i^{-2}} |1\rangle$ に変換する。
- iv) 補助量子ビットを測定し、逆演算（uncomputation）によって不要な量子ビットを $|0\rangle$ に戻しておく。測定結果が0のとき、残りの量子ビットの状態は、 $\sum_i \beta_i \lambda_i^{-1} |u_i\rangle = |A^{-1}\mathbf{b}\rangle$ が得られる（測定結果が1の場合にはステップ i) に戻る）。

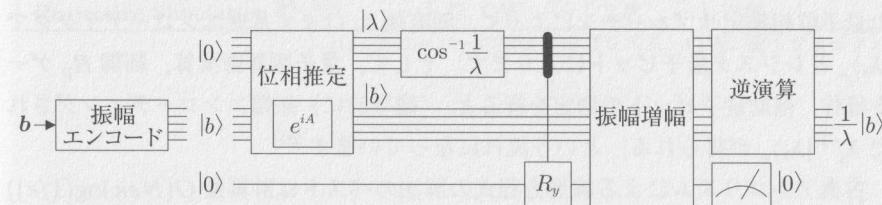


図4.10 HHL アルゴリズムの概要

*19 A が正則のとき、最大固有値 $\lambda_{\max}$ と最小固有値 $\lambda_{\min}$ の絶対値の比 $|\lambda_{\max}/\lambda_{\min}|$ です。